

Operating Systems and System Programming

Practical – 8

Aim

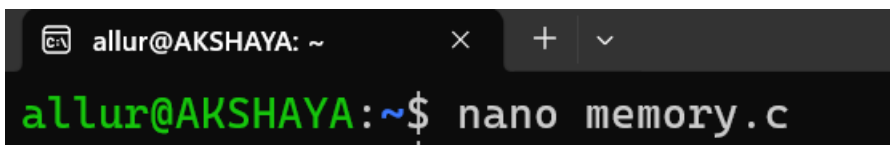
To demonstrate dynamic memory allocation using malloc(), calloc(), realloc(), and free(), identify memory leaks using Valgrind, and study Copy-on-Write (COW) behavior after fork() by analyzing memory usage before and after modifying data in the child process.

Algorithm

1. Start the program.
2. Allocate memory dynamically using malloc() and calloc().
3. Resize allocated memory using realloc() and initialize the data.
4. Display the allocated values and release memory using free().
5. Use Valgrind to check for memory leaks and memory errors.
6. Allocate memory and initialize data before creating a child process using fork().
7. Display memory usage before and after fork().
8. Modify the allocated data in the child process and observe the memory usage.
9. Verify that the parent's data remains unchanged.
10. Use wait() to synchronize the parent and child processes.
11. Release allocated memory and terminate the program.

Task-1:

Step-1:

A screenshot of a terminal window. The title bar shows a window icon, the text 'allur@AKSHAYA: ~', and window control buttons (close, maximize, and a dropdown arrow). The terminal content shows the prompt 'allur@AKSHAYA:~\$' followed by the command 'nano memory.c'.

Step-2:

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 memory.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int main()  
{  
    int *a, *b, i;  
  
    a = malloc(3 * sizeof(int));  
    if (a == NULL)  
        return 1;  
  
    for (i = 0; i < 3; i++)  
        a[i] = (i + 1) * 10;  
  
    b = calloc(3, sizeof(int));  
    if (b == NULL) {  
        free(a);  
        return 1;  
    }  
  
    a = realloc(a, 5 * sizeof(int));  
    if (a == NULL) {  
        free(b);  
        return 1;  
    }  
  
    a[3] = 40;  
  
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
    a[3] = 40;  
    a[4] = 50;  
  
    printf("malloc and realloc values:\n");  
    for (i = 0; i < 5; i++)  
        printf("%d ", a[i]);  
  
    printf("\ncalloc values:\n");  
    for (i = 0; i < 3; i++)  
        printf("%d ", b[i]);  
  
    free(a);  
    free(b);  
  
    printf("\nMemory released successfully.\n");  
    return 0;  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Ex  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Ju
```

Step-3:

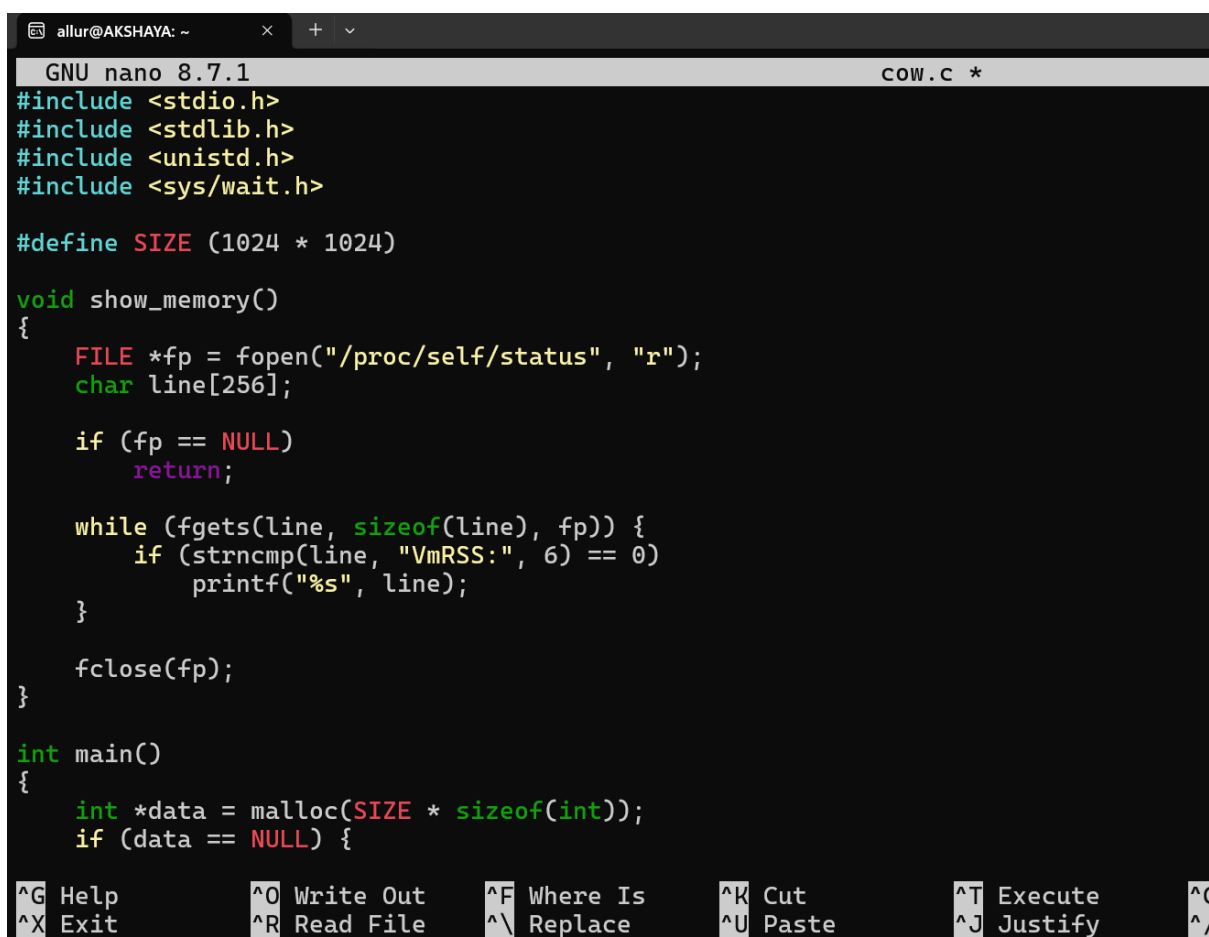
```
allur@AKSHAYA:~$ gcc memory.c -o memory
allur@AKSHAYA:~$ ./memory
malloc and realloc values:
10 20 30 40 50
calloc values:
0 0 0
Memory released successfully.
allur@AKSHAYA:~$ |
```

Task-2:

Step-1:

```
allur@AKSHAYA:~$ nano cow.c
```

Step-2:



```
GNU nano 8.7.1 COW.C *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (1024 * 1024)

void show_memory()
{
    FILE *fp = fopen("/proc/self/status", "r");
    char line[256];

    if (fp == NULL)
        return;

    while (fgets(line, sizeof(line), fp)) {
        if (strncmp(line, "VmRSS:", 6) == 0)
            printf("%s", line);
    }

    fclose(fp);
}

int main()
{
    int *data = malloc(SIZE * sizeof(int));
    if (data == NULL) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^/

```
allur@AKSHAYA: ~  
GNU nano 8.7.1 COW.C *  
if (data == NULL) {  
    perror("malloc");  
    return 1;  
}  
  
for (int i = 0; i < SIZE; i++)  
    data[i] = 10;  
  
printf("Parent PID: %d\n", getpid());  
printf("Memory before fork:\n");  
show_memory();  
  
pid_t pid = fork();  
  
if (pid < 0) {  
    perror("fork");  
    free(data);  
    return 1;  
}  
  
if (pid == 0) {  
    printf("\nChild PID: %d\n", getpid());  
    printf("Child memory before modification:\n");  
    show_memory();  
  
    data[0] = 100;  
  
    ^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C L  
    ^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ G
```

```
        data[0] = 100;  
  
        printf("Child modified data[0] = %d\n", data[0]);  
        printf("Child memory after modification:\n");  
        show_memory();  
  
        free(data);  
        return 0;  
    } else {  
        wait(NULL);  
  
        printf("\nParent data[0] = %d\n", data[0]);  
        printf("Parent memory after child modification:\n");  
        show_memory();  
  
        free(data);  
    }  
  
    return 0;  
}  
  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

Step-3:

```
allur@AKSHAYA:~$ gcc cow.c -o cow
allur@AKSHAYA:~$ ./cow
Parent PID: 608
Memory before fork:
VmRSS:          5908 kB

Child PID: 609
Child memory before modification:
VmRSS:          5100 kB
Child modified data[0] = 100
Child memory after modification:
VmRSS:          5100 kB

Parent data[0] = 10
Parent memory after child modification:
VmRSS:          5908 kB
allur@AKSHAYA:~$
```

Result

Dynamic memory allocation was demonstrated using malloc(), calloc(), realloc(), and free(). Memory leaks were checked using Valgrind. Copy-on-Write behavior was demonstrated using fork(), showing that the child's modifications do not affect the parent's data.